COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 8, 2026	**DATE SUBMITTED:**	September 10, 2026
NAME:	Niña Denise M. Santiago	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 7.1 — **Sorting Algorithms Pt1**

1. PROCEDURE OUTPUT

Code + Console Screenshot

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sorting.h"

const int max_size = 100;

int main (){
    int testArray[max_size];

    srand(time(0)); //Seed the random number generator

    //Fills array with 100 random numbers
    for (int i = 0; i < max_size; i++) {
        testArray[i] = std::rand() % 100 + 1; //std::rand = random number gen
    }

    //DISPLAYING THE ARRAY
    std::cout<<"Array elements:\n";
    displayArray(testArray, max_size);
```

sorting.h > BubbleSort<T>(T [], int)

```cpp
#ifndef SORTING_H
#define SORTING_H

#include <iostream>

//DISPLAYING ALL ARRAY ELEMENTS
void displayArray(const int values[], int maxSize) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << values[i] << ' ';

        if ((i + 1) % 10 == 0) {
            std::cout << "";
        }
    }
}
```

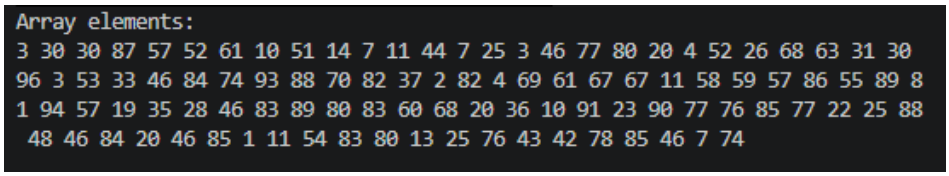
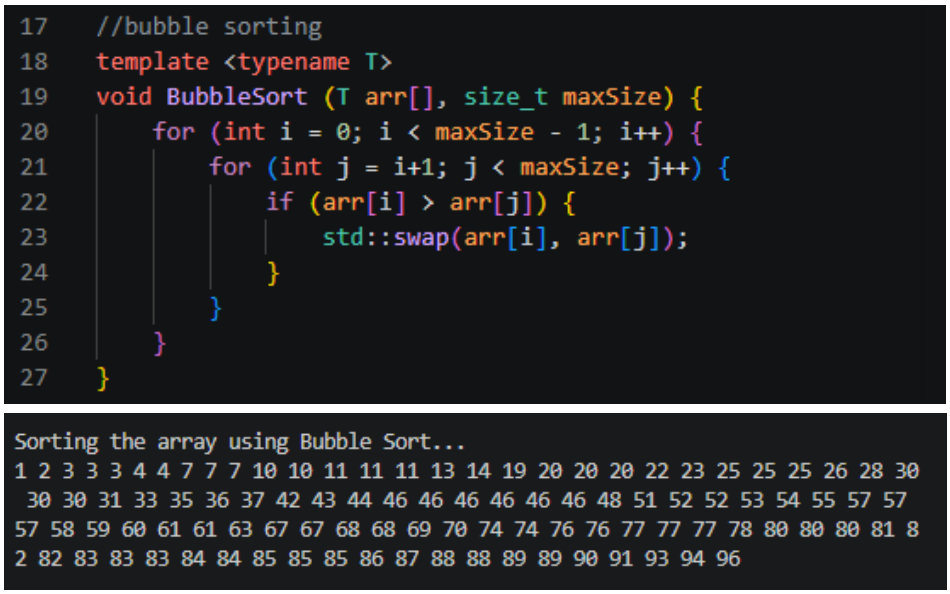
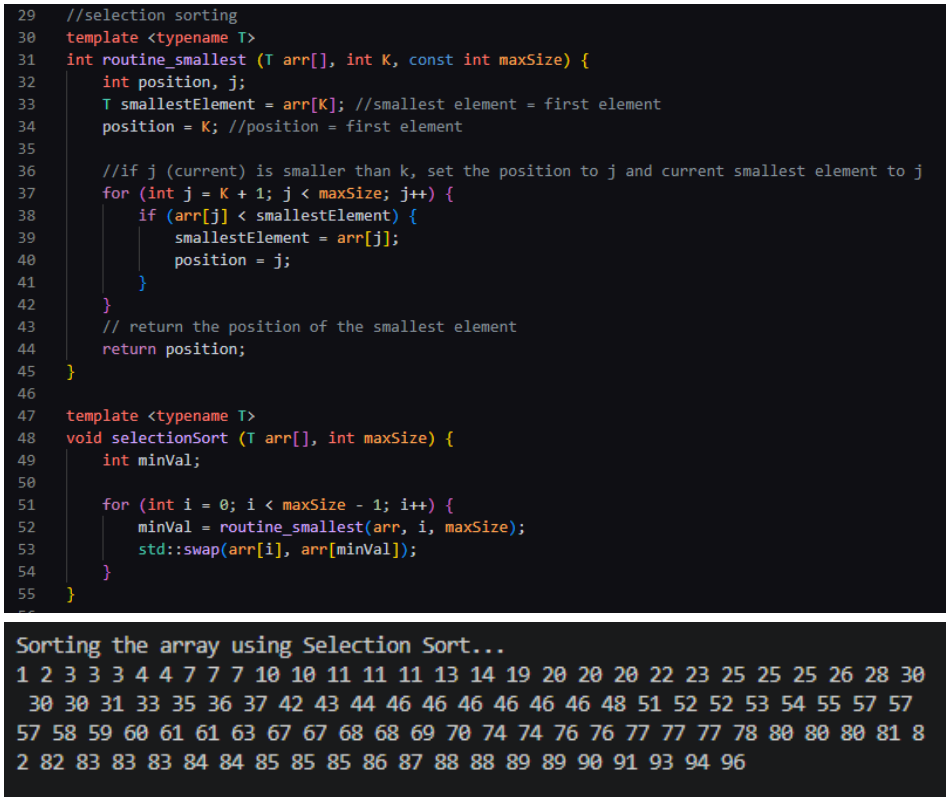
	
Observations	The initial array contains 100 unsorted integer values. The values are arranged randomly, so there is no specific order from the smallest to the largest number. Since the array contains 100 elements, the sorting algorithms need to process and compare these values to arrange them correctly. The initial state of the array shows that sorting is necessary before the values can be easily analyzed. The generation and display of the array take O(n) time because each element is processed once.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	The Bubble Sort code uses two nested loops. The outer loop goes through each element, while the inner loop compares the current element with the rest of the array. If an element is larger than the next one, they are swapped. This process continues until the array is sorted. The algorithm relies heavily on repeated comparisons and swaps, which makes it simple but not very efficient. Bubble Sort has a worst-case and average-case time complexity of $O(n^2)$ because every element may need to be compared with every other element. In the best case, when the array is already sorted, it

	can run in O(n).

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	
Observations	The code works by repeatedly finding the smallest element in the unsorted part of the array and swapping it with the first unsorted element. The function routine_smallest identifies the position of the smallest element, and then the main loop swaps it into place. This reduces the number of swaps compared to Bubble Sort but still requires many comparisons. Selection Sort has a time complexity of $O(n^2)$ in all cases (best, average, and worst). This is because it always scans the remaining unsorted elements to find the minimum, even if the array is already sorted.

```
//selection sorting
template <typename T>
int routine_smallest (T arr[], int K, const int maxSize) {
    int position, j;
    T smallestElement = arr[K]; //smallest element = first element
    position = K; //position = first element

    //if j (current) is smaller than k, set the position to j and current smallest element to j
    for (int j = K + 1; j < maxSize; j++) {
        if (arr[j] < smallestElement) {
            smallestElement = arr[j];
            position = j;
        }
    }
    // return the position of the smallest element
    return position;
}

template <typename T>
void selectionSort (T arr[], int maxSize) {
    int minVal;

    for (int i = 0; i < maxSize - 1; i++) {
        minVal = routine_smallest(arr, i, maxSize);
        std::swap(arr[i], arr[minVal]);
    }
}
```

```
Sorting the array using Selection Sort...
1 2 3 3 3 4 4 7 7 7 10 10 11 11 11 13 14 19 20 20 20 22 23 25 25 25 26 28 30
 30 30 31 33 35 36 37 42 43 44 46 46 46 46 46 46 48 51 52 52 53 54 55 57 57
57 58 59 60 61 61 63 67 67 68 68 69 70 74 74 76 76 77 77 77 78 80 80 80 81 8
2 82 83 83 83 84 84 85 85 85 86 87 88 88 89 89 90 91 93 94 96
```

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot

```
57  //insertion sorting
58  template <typename T>
59  void InsertionSort (T arr[], int maxSize) {
60      int k = 1, j;
61      T temp; //swapping, cannot use std::swap
62
63      while (k < maxSize) {
64          temp = arr[k];
65          j = k - 1;
66
67          while (j >= 0 && temp <= arr[j]){
68              arr[j + 1] = arr[j];
69              j--; //backtracking part
70          }
71          //reassigning values and moving to next element
72          arr[j + 1] = temp;
73          k++;
74      }
75  }
```

```
Sorting the array using Insertion Sort...
1 2 3 3 3 4 4 7 7 7 10 10 11 11 11 13 14 19 20 20 20 22 23 25 25 25 26 28 30
 30 30 31 33 35 36 37 42 43 44 46 46 46 46 46 46 48 51 52 52 53 54 55 57 57
57 58 59 60 61 61 63 67 67 68 68 69 70 74 74 76 76 77 77 77 78 80 80 80 81 8
2 82 83 83 83 84 84 85 85 85 86 87 88 88 89 89 90 91 93 94 96
```

Observations

In this code, each new element is compared with the elements before it, and larger values are moved one position forward until the correct position for the current element is found. The sorted output shows that the algorithm correctly arranged all the values. This method is useful because it builds the sorted portion of the array gradually instead of searching the entire remaining array for the smallest value. The code uses a while loop to backtrack and place the element in its proper position. This makes it efficient for small or nearly sorted arrays. Its average and worst-case time complexity is $O(n^2)$ because elements may need to be shifted several times. However, its best-case time complexity is O(n) when the array is already or nearly sorted.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code + Console Screenshot

suppAct.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sorting.h"

const int max_size = 100;

int main (){
    int testArray[max_size];

    // CANDIDATE LIST
    std::string candidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustín",
        "Deja Layla Bañaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    // DISPLAYING THE CANDIDATES
    std::cout << "===== LIST OF CANDIDATES =====\n";

    for (int i = 0; i < 5; i++) {
        std::cout << "Candidate " << i + 1 << ": "
                  << candidates[i] << "\n";
    }

    std::cout << "\n";

    srand(time(0)); // Seed the random number generator

    //GENERATING THE VOTES
    for (int i = 0; i < max_size; i++) {
        testArray[i] = std::rand() % 5 + 1; // Values represent candidates 1 to 5
    }

    //DISPLAYING THE VOTES(ARRAY)
    std::cout<<"Array elements:\n";
    displayArray(testArray, max_size);

    //SORTING THE VOTES
    std::cout<<"\n\nSorting the votes using Selection Sort...\n";
    selectionSort(testArray, max_size);
    displayArray(testArray, max_size);

    int winnerCand = 1;
    int highestVotes = 0;
```

```cpp

    //SEARCHING FOR EACH CANDIDATE'S VOTES
    std::cout << "\n\nVote count:\n";

    for (int candidate = 1; candidate <= 5; candidate++) {
        int votes = CountVotes(testArray, max_size, candidate);

        std::cout << "Candidate " << candidate << " - "
                  << candidates[candidate - 1] << ": "<< votes << " votes\n";

        if (votes > highestVotes) {
            highestVotes = votes;
            winnerCand = candidate;
        }
    }

    // DISPLAYING THE WINNING CANDIDATE
    std::cout << "\nWinner: Candidate " << winnerCand;
    std::cout << " with " << highestVotes << " votes\n";

    return 0;
}
```

```cpp
//selection sorting
template <typename T>
int routine_smallest (T arr[], int K, const int maxSize) {
    int position, j;
    T smallestElement = arr[K]; //smallest element = first element
    position = K; //position = first element

    //if j (current) is smaller than k, set the position to j and current smallest element to j
    for (int j = K + 1; j < maxSize; j++) {
        if (arr[j] < smallestElement) {
            smallestElement = arr[j];
            position = j;
        }
    }
    // return the position of the smallest element
    return position;
}

template <typename T>
void selectionSort (T arr[], int maxSize) {
    int minVal;

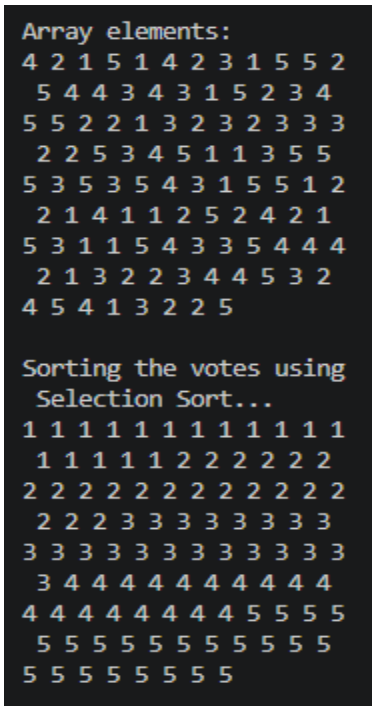
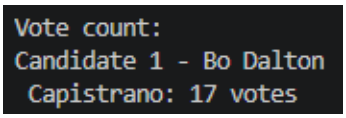
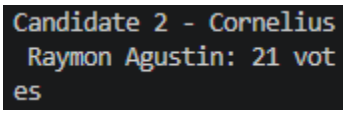
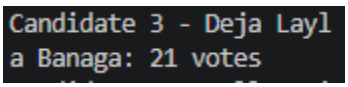
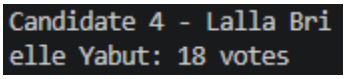
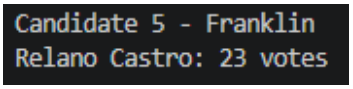
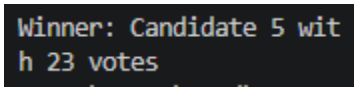
    for (int i = 0; i < maxSize - 1; i++) {
        minVal = routine_smallest(arr, i, maxSize);
        std::swap(arr[i], arr[minVal]);
    }
}
```

```cpp
//======================SUPPLEMENTARY=========================
//searches for the number of votes
template <typename T>
int CountVotes (const T arr[], int maxSize, T candidate) {
    int votes = 0;

    for (int i = 0; i < maxSize; i++) {
        if (arr[i] == candidate) {
            votes++;
        }
    }

    return votes;
}
```

Observations	The program generates random votes for five candidates, sorts them using Selection Sort, and then counts the votes using the CountVotes function. The function loops through the array and increments a counter whenever it finds a match for the candidate number. This ensures that each candidate's votes are correctly tallied.It then compares the vote totals and stores the candidate with the highest number of votes as the winning candidate. The use of the candidate array also allows the program to display the candidates' names together with their vote counts. The CountVotes() function has a time complexity of O(n) for one candidate because it checks every element in the array once. Since the program checks five candidates, the overall counting process is O(5n), which simplifies to O(n) because 5 is a constant. I chose Selection Sort because it is simple to implement and appropriate for the size of the array used in this activity. It arranges the vote values in ascending order by repeatedly finding the smallest value and placing it in the correct position. Sorting the values also groups the same candidate numbers together, making the vote-counting process easier to analyze.

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	**Candidate 1:** 17	
	Candidate 2: 21	
	Candidate 3: 21	
	Candidate 4: 18	
	Candidate 5: 23	
		

Question: Was your developed vote counting algorithm effective? Why or why not?

Answer: Yes, the developed vote-counting algorithm was effective because it correctly counted the number of votes received by each candidate and identified the candidate with the highest number of votes. The CountVotes() function I made also checks each element in the array and increases the vote count when the value matches the candidate being searched. After counting all five candidates, the program compares their vote totals and determines the winner. The use of Selection Sort in my program also helped organize the vote values before counting. Therefore, the algorithm was effective because it produced the expected output I need for vote counts and correctly determined the winning candidate.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.